Aplikacja do biometrycznego przetwarzania obrazu tęczówki

Aleksandra Zawadka Jan Taran

22 kwietnia 2026

Spis treści

1 Wstęp i opis aplikacji **2**
1.1 Wykorzystane technologie i biblioteki 2

2 Układ interfejsu użytkownika **2**

3 Opis działania programu **2**
3.1 Krok 0: Wczytanie zdjęcia 2
3.2 Krok 1: Konwersja do skali szarości 3
3.3 Krok 2: Detekcja źrenicy (Binaryzacja) 3
3.4 Krok 3: Morfologia źrenicy 3
3.5 Krok 4: Wyznaczenie środka i promienia źrenicy 3
3.6 Krok 5: Detekcja tęczówki (Binaryzacja) 3
3.7 Krok 6: Morfologia tęczówki 3
3.8 Krok 7: Wyznaczenie promienia tęczówki 4
3.9 Krok 8: Rozwinięcie tęczówki (Normalizacja) 4
3.10 Krok 9: Kodowanie Daugmana (Gabor) 4
3.11 Krok 10: Moduł porównawczy (Weryfikacja) 4

4 Ewaluacja systemu **4**
4.1 Wpływ artefaktów wizualnych na proces segmentacji 4
4.2 Skuteczność wyodrębniania 5
4.3 Weryfikacja tożsamości i analiza stabilności kodu 6

1 Wstęp i opis aplikacji

Celem projektu była realizacja aplikacji okienkowej przeznaczonej do biometrycznego rozpoznawania osób na podstawie obrazu tęczówki oka. System umożliwia pełną, interaktywną wizualizację procesu przetwarzania, obejmującą segmentację źrenicy i tęczówki, normalizację obrazu do postaci prostokątnej zgodnie z algorytmem Daugmana oraz ekstrakcję cech przy użyciu falek Gabora. Aplikacja pozwala na wygenerowanie unikalnego kodu tęczówki oraz jego weryfikację poprzez obliczenie odległości Hamminga względem innych wzorców.

1.1 Wykorzystane technologie i biblioteki

Do budowy aplikacji wykorzystano następujące środowiska i biblioteki:

- **PyQt5** - Posłużyła do stworzenia GUI. Wybór podyktowany był dużymi możliwościami personalizacji, wsparciem dla architektury sygnałów i slotów oraz wysoką stabilnością aplikacji okienkowych.
- **NumPy** - Wszystkie operacje na pikselach, filtry przestrzenne, sploty oraz operacje morfologiczne zostały zaimplementowane z użyciem wektoryzacji i zoptymalizowanych operacji macierzowych, co znacząco przyspieszyło działanie programu.
- **pyqtgraph** - Wykorzystana do dynamicznej wizualizacji projekcji horyzontalnych i wertykalnych, używanych w procesie wyznaczania geometrycznego środka źrenicy.
- **Pillow (PIL)** - Zastosowana do operacji szybkiego odczytu i zapisu plików graficznych (w tym zapisywania wygenerowanych kodów jako map bitowych BMP).

2 Układ interfejsu użytkownika

Interfejs graficzny aplikacji został zaprojektowany w sposób modułowy, oparty na systemie kolejnych "kroków". Pozwala to na przejrzystą prezentację i izolację kolejnych etapów przetwarzania biometrycznego.

W lewym górnym rogu okna umieszczono przycisk umożliwiający wczytanie obrazu wejściowego z dysku. W centralnej części widoku zlokalizowano obszar roboczy wyświetlający aktualnie przetwarzane zdjęcie. Pod nim znajduje się interaktywny panel, w którym - w zależności od aktualnego kroku - pojawiają się suwaki służące do doboru kluczowych parametrów (np. progów binaryzacji czy wielkości masek morfologicznych).

W dolnej części okna umieszczono panel nawigacyjny z przyciskami "Wstecz" oraz "Dalej", pozwalający użytkownikowi na swobodne przemieszczanie się wzdłuż osi czasu algorytmu i obserwowanie efektów wprowadzanych modyfikacji na żywo. W przedostatnim kroku użytkownik ma możliwość trwałego zapisu wygenerowanego, znormalizowanego kodu tęczówki do pliku. Ostatni etap aktywuje dedykowany panel porównawczy, służący do zestawiania par kodów na bazie odległości Hamminga.

3 Opis działania programu

Aplikacja przeprowadza proces w dziesięciu etapach. Rozwiązanie to ma na celu dogłębną analizę każdego ze zjawisk zachodzących podczas ekstrakcji cech osobniczych z surowej fotografii.

3.1 Krok 0: Wczytanie zdjęcia

Aplikacja ładuje surowy obraz wejściowy. Obraz ten jest standaryzowany do macierzy RGB (Numpy Array), aby zapewnić spójność przetwarzania niezależnie od specyfikacji formatu pliku graficznego.

3.2 Krok 1: Konwersja do skali szarości

Obraz kolorowy zostaje przekształcony na odcienie szarości przy użyciu klasycznych wag luminancji. Eliminacja kanałów barwnych redukuje złożoność obliczeniową dla dalszych kroków detekcji strukturalnej:

$$A(i,j) = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \tag{1}$$

W tym kroku wyznaczany jest również bazowy próg jasności P, będący uśrednioną intensywnością wszystkich pikseli obrazu o wysokości h i szerokości w:

$$P = \frac{1}{h \cdot w} \sum_{i=0}^{h-1} \sum_{j=0}^{w-1} A(i,j) \tag{2}$$

3.3 Krok 2: Detekcja źrenicy (Binaryzacja)

Źrenica, będąca najciemniejszym obszarem oka, wyodrębniana jest przy pomocy algorytmu progowania. Wartość progu odcięcia P_I obliczana jest na podstawie suwaka parametrycznego x_I dostępnego w interfejsie graficznym:

$$P_I = \frac{P}{x_I} \tag{3}$$

3.4 Krok 3: Morfologia źrenicy

Binaryzacja wyłapuje obiekty o podobnej jasności co źrenica, co prowadzi do pojawiania się rzęs, a refleksy świetlne tworzą dziury w masce. Aplikacja wyposażona jest w interaktywny panel operacji morfologicznych bazujących na szybkich filtrach *min/max* przesuwnego okna. Użytkownik może w czasie rzeczywistym dobierać rodzaj i rozmiar filtra, skutecznie "zalewając" refleksy świetlne i eliminując niepożądane wypustki, uzyskując idealnie okrągłą, izolowaną maskę źrenicy.

```
# Przyklad operacji otwarcia w kodzie (usuniecie rzes)
tmp = IrisProcessor.filter_min_max(img, size, 'max')
img = IrisProcessor.filter_min_max(tmp, size, 'min')
```

3.5 Krok 4: Wyznaczenie środka i promienia źrenicy

Środek (c_x, c_y) oraz promień źrenicy wyznaczane są na wyczyszczonym w poprzednim kroku obrazie binarnym. Metoda opiera się na sumowaniu wektorów rzutowanych wzdłuż zróżnicowanych osi (projekcje wertykalne, horyzontalne oraz skośne). Wynikiem jest wycentrowany czerwony celownik i okrąg okalający maskę źrenicy.

3.6 Krok 5: Detekcja tęczówki (Binaryzacja)

Tęczówka, będąca jaśniejsza od źrenicy, ale znacznie ciemniejsza od twardówki, izolowana jest za pomocą drugiego, dedykowanego progu P_P:

$$P_P = \frac{P}{x_P} \tag{4}$$

3.7 Krok 6: Morfologia tęczówki

Podobnie jak przy źrenicy, zbinaryzowana tęczówka zazwyczaj łączy się z cieniami powiek i górną warstwą rzęs. Kolejny moduł morfologiczny pozwala na "oderwanie" maski tęczówki od otaczających struktur, pozostawiając idealny, duży okrąg, co zapobiega przekłamaniom przy wyliczaniu jej zewnętrznych granic.

3.8 Krok 7: Wyznaczenie promienia tęczówki

Ze względu na fakt, iż oko ludzkie niemal zawsze jest osłonięte powieką od góry i dołu, rzutowanie na cały obraz generuje błędne wyniki. Algorytm omija to zjawisko, wycinając i uśredniając wyłącznie wąski, poziomy pas przechodzący przez wcześniej wyznaczony środek źrenicy. Gradient (największy skok jasności) w obrębie tego pasa stanowi niezaburzoną lewą i prawą granicę promienia tęczówki.

3.9 Krok 8: Rozwinięcie tęczówki (Normalizacja)

Wydobyty pierścień tęczówki wymaga normalizacji wymiarowej. Dokonuje się tego przy użyciu modelu "gumowej płachty" (ang. *rubber sheet model*) Johna Daugmana. Promieniowe współrzędne mapowane są do stałego prostokąta, niezależnego od zwężenia/rozszerzenia samej źrenicy na badanej fotografii.

$$x = c_x + r \cdot \cos(\theta), \quad y = c_y + r \cdot \sin(\theta) \tag{5}$$

Algorytm pobiera dane wyłącznie z dwóch bezpiecznych stref kątowych (z ominięciem newralgicznych biegunów góra-dół obarczonych okluzją rzęs).

3.10 Krok 9: Kodowanie Daugmana (Gabor)

Rozwinięta, prostokątna tęczówka dzielona jest na poziome pasy, na których dokonywana jest dekompozycja strukturalna za pomocą filtru 1D opisywanego funkcją Gabora. Szerokość dzwonu Gaussa regulowana jest parametrem σ:

$$\sigma = \frac{1}{2}\pi f \tag{6}$$

W tym kroku wprowadzono dwie ważne poprawki algorytmiczne. Po pierwsze, od sygnału odjęto jego uśrednioną wartość, co sprawiło, że oscyluje on teraz naturalnie wokół zera. Po drugie, na lewej i prawej krawędzi obrazu dodano marginesy wykorzystujące 'odbicie lustrzane'. Zabiegi te całkowicie wyeliminowały problem sztucznych krawędzi, na których filtry matematyczne wcześniej generowały fałszywe wyniki. Tak oczyszczony sygnał - bazujący na wybranej z suwaka częstotliwości fali (f) - jest ostatecznie zamieniany na ciąg zer i jedynek, tworząc unikalny kod biometryczny *IrisCode*.

3.11 Krok 10: Moduł porównawczy (Weryfikacja)

Ostatni etap aplikacji to niezależny moduł służący do weryfikacji tożsamości. Użytkownik może w nim wczytać dwa zapisane wcześniej kody w formacie BMP. Algorytm wczytuje te obrazy jako binarne macierze i wykonuje na nich operację logiczną XOR (Exclusive OR). Następnie wyliczana jest Odległość Hamminga, będąca stosunkiem liczby różniących się bitów do wszystkich bitów kodu. Na podstawie tej wartości (przy progu decyzyjnym ustawionym na 0.3) system automatycznie ocenia, czy załadowane kody należą do tej samej osoby.

4 Ewaluacja systemu

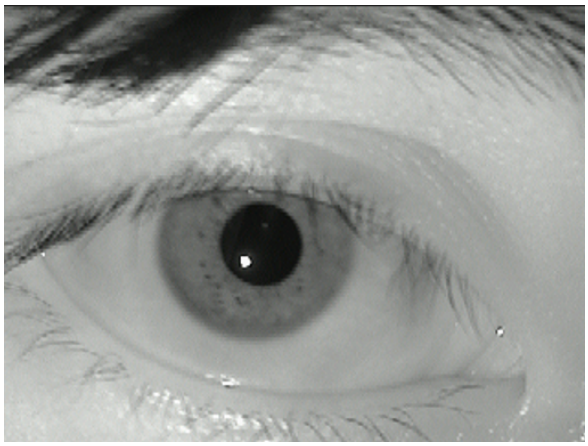
Ewaluację pracy przeprowadzono na publicznie dostępnym zbiorze danych testowych, analizując zarówno przypadki pozytywne, jak i ograniczające detekcję zakłócenia fizyczne.

4.1 Wpływ artefaktów wizualnych na proces segmentacji

Największym wyzwaniem dla zaimplementowanego systemu są czynniki zewnętrzne przysłaniające samą tęczówkę. Ze względu na fakt, że algorytmy detekcji środka opierają się na profilu

intensywności sumowanych pikseli, obiekty o barwie zbliżonej do samej źrenicy są wchłaniane do maski binarnej w pierwszych krokach operacji.

Najpowszechniejszym problemem okazały się opadające ciemne włosy wchodzące w kadr. Zniekształcają one maskę geometryczną do tego stopnia, że nawet najostrzejsze filtry otwarcia morfologicznego nie są w stanie sprowadzić jej do idealnego okręgu (zob. Rysunek 1).

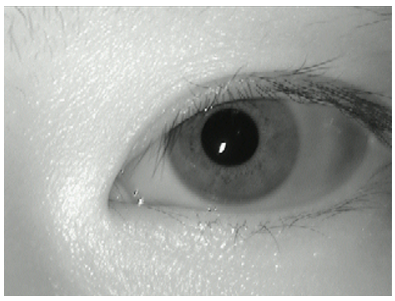


(a) Obraz wejściowy z włosami

(b) Maska - widoczne wciąż włosy

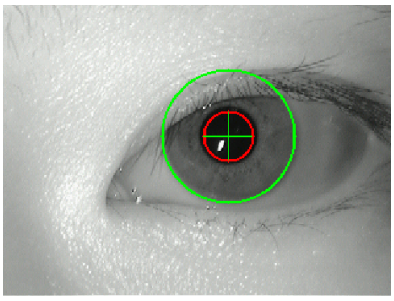
Rysunek 1: Nieskuteczność operacji morfologicznych w przypadku rozległej okluzji przez ciemne włosy o jasności zbliżonej do źrenicy.

Kolejnym ograniczeniem są okluzje powiek na nieotwartym do końca oku oraz cienie padające na twardówkę. Jak można zauważyć, artefakty te znacząco utrudniają proces czyszczenia, a wygenerowana maska morfologiczna daleka jest od ideału (zlewając się z cieniem z prawej strony). Mimo to, zaimplementowany w systemie algorytm detekcji promienia wykazuje w takich warunkach znakomitą odporność. Dzięki zawężeniu analizy gradientu jasności wyłącznie do wąskiego, poziomego pasa przechodzącego przez środek oka, system skutecznie ignoruje zakłócenia z góry i z dołu, poprawnie wyznaczając okręgi źrenicy i tęczówki (zob. Rysunek 2).



(a) Obraz wejściowy

(b) Zaburzona morfologia

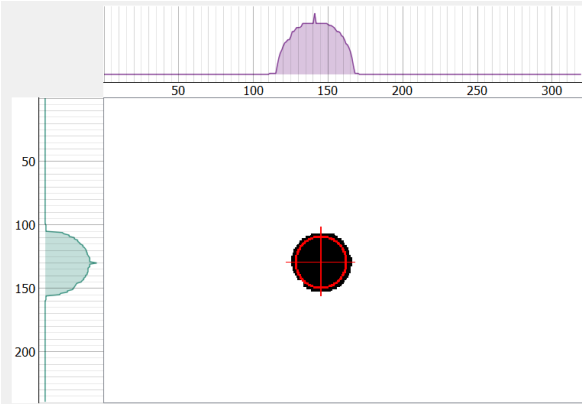


(c) Poprawne promienie

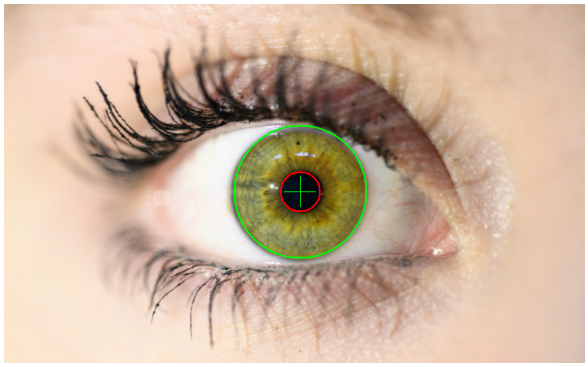
Rysunek 2: Odporność algorytmu lokalizacyjnego na głębokie cienie i przysłonięcie powieką, pomimo silnie zniekształconej maski binarnej.

4.2 Skuteczność wyodrębniania

W obrazach optymalnych (przy otwartym oku i prawidłowym doświetleniu), system cechuje się dużą odpornością na drobny szum. Moduł morfologiczny bezbłędnie pozbywa się refleksów okien i żarówek widocznych na powierzchni źrenicy, co doskonale obrazuje Rysunek 3.



(a) Identyfikacja środka i promienia źrenicy

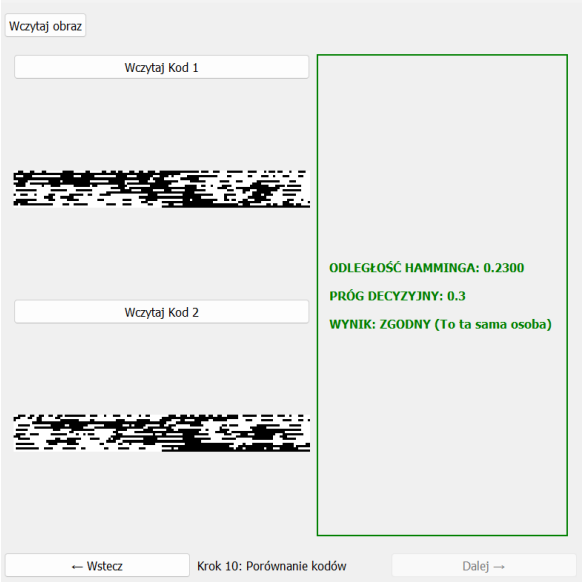


(b) Odcięcie tęczówki od twardówki

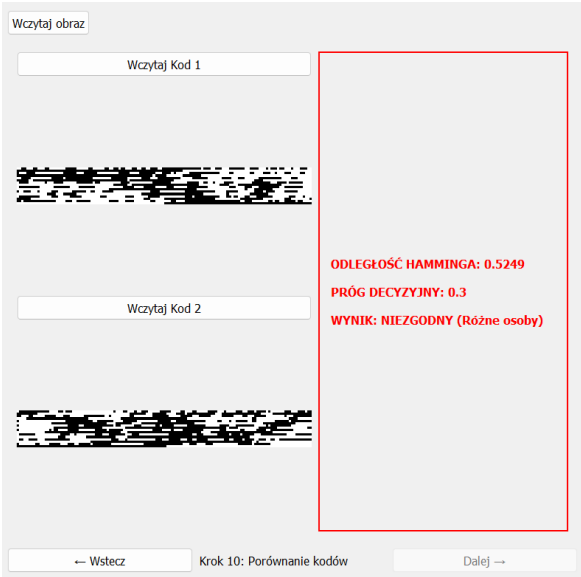
Rysunek 3: Prawidłowe działanie algorytmów lokalizacyjnych i projekcji na oczyszczonym obrazie.

4.3 Weryfikacja tożsamości i analiza stabilności kodu

Ostatni etap aplikacji (Krok 10) to dedykowany moduł służący do weryfikacji tożsamości. Algorytm wczytuje dwa zapisane wcześniej kody w formacie BMP jako binarne macierze i wykonuje na nich operację logiczną XOR (Exclusive OR). Następnie wyliczana jest Odległość Hamminga, będąca stosunkiem liczby różniących się bitów do całkowitej liczby bitów w wygenerowanym kodzie. Na podstawie tej wartości system ocenia, czy załadowane wzorce należą do tej samej osoby. Przykładowe ekrany działania tego modułu w warunkach prawidłowej akceptacji oraz poprawnego odrzucenia zaprezentowano na Rysunku 4.



(a) Zgodność (to samo oko, różne ujęcia)



(b) Brak zgodności (oko lewe i prawe)

Rysunek 4: Przykładowe ekrany modułu porównawczego dla dwóch skrajnych wyników odległości Hamminga.

W celu oceny skuteczności systemu przeprowadzono serię testów porównawczych na bazie zdjęć z bazy MMU Iris Database. Proces weryfikacji oparto na ujęciach tego samego oka (testy wewnątrzklasowe), porównaniu oka lewego z prawym u tej samej osoby oraz zestawieniu z tęczówkami osób trzecich (testy międzyklasowe). Wyniki analizy dla progu decyzyjnego równego 0.30 zestawiono w Tabeli 1.

Tabela 1: Szczegółowe wyniki odległości Hamminga badanych próbek (Próg decyzyjny: 0.30).

Scenariusz testowy	**Wyniki cząstkowe (L1 vs...)**	**Średnia**	**Werdykt**
To samo oko (lewe)	L2: 0.2627, L3: 0.2695, L4: 0.2871, L5: 0.3193	**0.2846**	ZGODNY
Oko lewe vs prawe	R1: 0.5063, R2: 0.4771, R3: 0.5063, R4: 0.5020, R5: 0.5186	**0.5020**	NIEZGODNY
Inne osoby (lewe oko)	Os.A: 0.3828, Os.B: 0.4126, Os.C: 0.4102, Os.D: 0.4204, Os.E: 0.3936	**0.4039**	NIEZGODNY

Podczas testów wewnątrzklasowych pominięto porównanie zdjęcia L1 z samym sobą, gdyż deterministyczny charakter algorytmu zawsze zwraca w tym przypadku dystans 0.0. Uzyskana średnia na poziomie około 0.28 pozwala na akceptację użytkownika. Warto zauważyć, że dla ujęcia L5 odległość nieznacznie przekroczyła próg (0.3193).

Niezwykle istotnym i poprawnym wynikiem jest porównanie oka lewego z prawym u tej samej osoby. Średni dystans wynoszący 0.50 (zob. Rysunek 4b) stanowi dowód biologicznej unikalności struktury oka - wzorce lewej i prawej tęczówki są od siebie statystycznie niezależne i osiągają maksymalną odległość Hamminga, typową dla dwóch całkowicie obcych sobie osób.

Z kolei porównanie badanej tęczówki z oczami obcych osób wygenerowało stabilną średnią na poziomie 0.40. Przesunięcie średniej z idealnego 0.50 do 0.40 wynika najprawdopodobniej ze wspólnych, powtarzalnych artefaktów oświetleniowych w bazie danych lub podobnego układu rzęs u różnych pacjentów, co zostało skutecznie skompensowane przez ustalony próg decyzyjny 0.30.

Literatura

[1] dr inż. Janusz Rafałko, Materiały dydaktyczne do przedmiotu Biometria, `https://pages.mini.pw.edu.pl/~rafalkoj/www/?Dydaktyka:2025%2F2026:-_Biometria` [dostęp: 22 kwietnia 2026].

[2] Zbiór danych "MMU Iris Database", Multimedia University, Malezja.